

Python Numeric Types, Parallel Processing

DS 5110: Big Data Systems

Summer 2026

Lecture 3

Yue Cheng



UNIVERSITY
of VIRGINIA

Some material taken/derived from:

- Wisconsin CS 544 and CS 320 by Tyler Caraza-Harter.

@ 2026 released for use under a [CC BY-SA](#) license.

Outline

- Python numeric data types
- Python parallel processing

Python numeric types (built in)

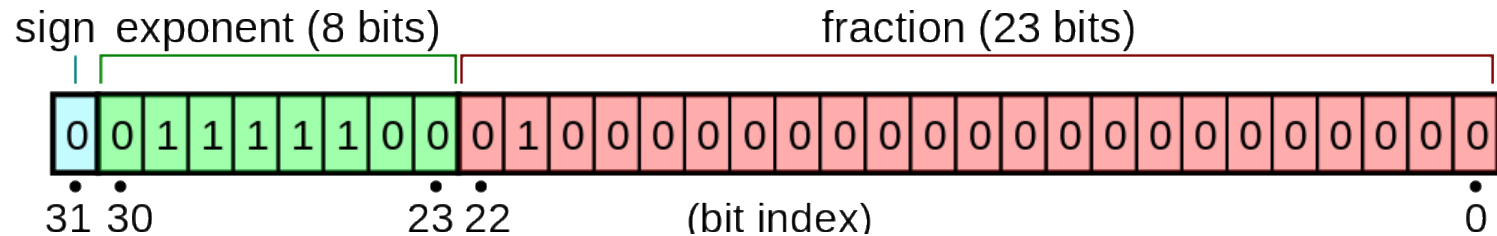
<https://docs.python.org/3/library/stdtypes.html#numeric-types-int-float-complex>

Python numeric types

- int
 - No max/min size (Python is unusual in this way)
 - Bigger values -> more bits necessary
- float
 - Defaults 64 bits (double precision)
 - You can also use float32 given a certain framework (e.g., PyTorch, etc.)
 - Historically, pre-trained ML models use **float32** for parameters
 - For modern AI (LLMs), **bfloat16** is very common now

float32

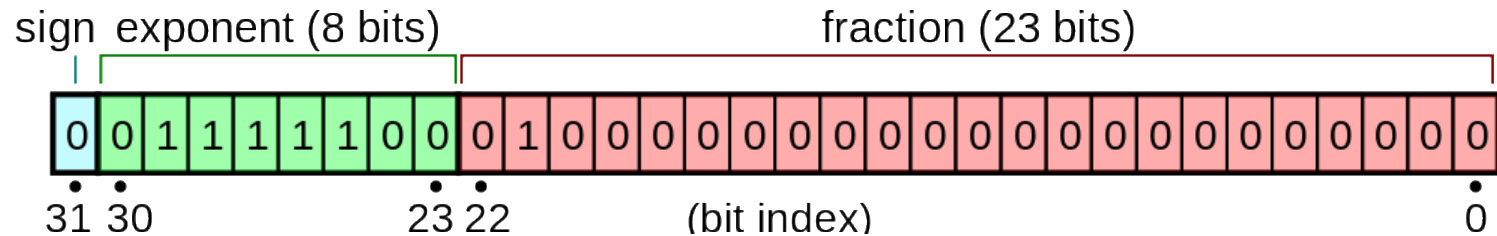
- Standard IEEE format (float32)



$$p = (-1)^s \times 2^{e-127} \times (1.m_1m_2 \dots m_{23})_2$$
$$= (-1)^s \times 2^{e-127} \times \left(1 + \sum_{i=1}^{23} m_i \times 2^{-i}\right)$$

float32

- Standard IEEE format (float32)



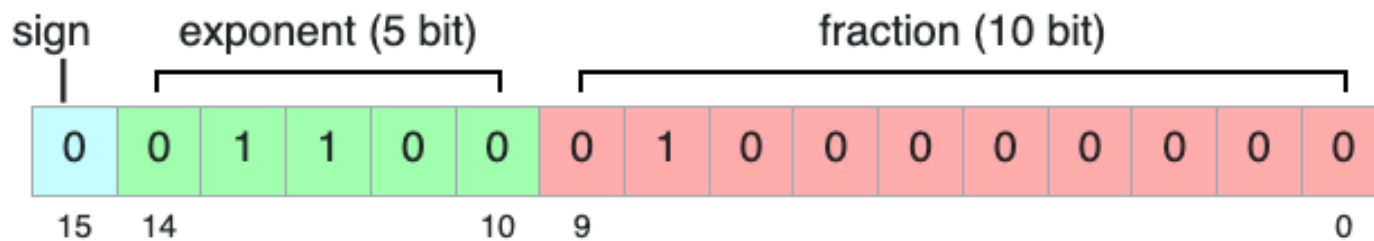
$$p = (-1)^s \times 2^{e-127} \times (1.m_1m_2 \dots m_{23})_2$$

$$= (-1)^s \times 2^{e-127} \times \left(1 + \sum_{i=1}^{23} m_i \times 2^{-i}\right)$$

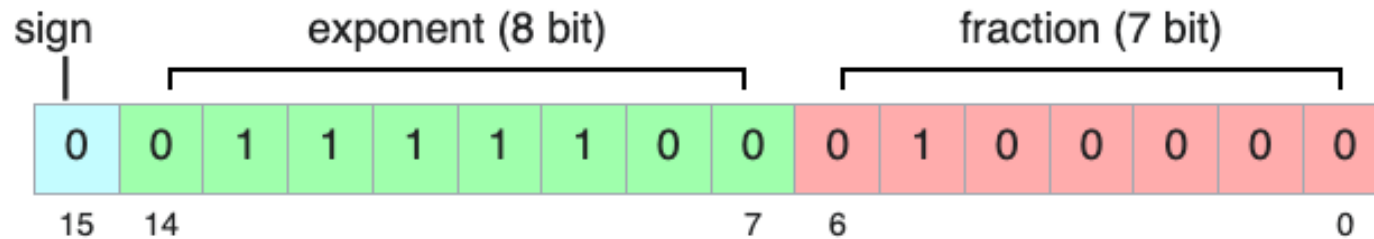
$$(-1)^0 \times 2^{124-127} \times (1 + 1 \cdot 2^{-2}) = (1/8) \times (1 + (1/4)) = 0.15625$$

float16 and bfloat16

float16



bfloat16



Python numeric types (built in)

<https://docs.python.org/3/library/stdtypes.html#numeric-types-int-float-complex>

Python numeric types

- int
 - No max/min size (Python is unusual in this way)
 - Bigger values -> more bits necessary
- float
 - Defaults 64 bits (double precision)
 - You can also use float32 given a certain framework (e.g., PyTorch, NumPy, etc.)
 - For LLMs, **bfloat16** is very common now
 - Min/max, inf, -inf, nan have special bit combinations

Python numeric types (built in)

<https://docs.python.org/3/library/stdtypes.html#numeric-types-int-float-complex>

Python numeric types

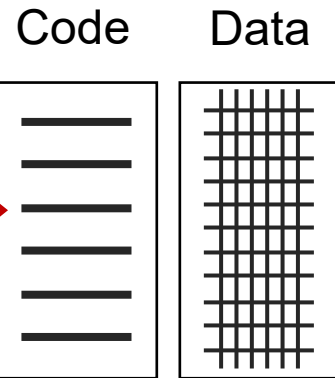
- int
 - No max/min size (Python is unusual in this way)
 - Bigger values -> more bits necessary
- float
 - Defaults 64 bits (double precision)
 - You can also use float32 given a certain framework (e.g., PyTorch, numpy, etc.)
 - For LLMs, **bfloat16** is very common now
 - Min/max, inf, -inf, nan have special bit combinations
- complex

Other (commonly used) numeric types

- Common numeric types that popular Python frameworks (e.g., PyTorch, NumPy) support
 - ints: uint8, int8, int16, int32, int64
 - floats: float16, bfloat16, float32, float64
 - dtype (data type): metadata that specifies the element type of a tensor

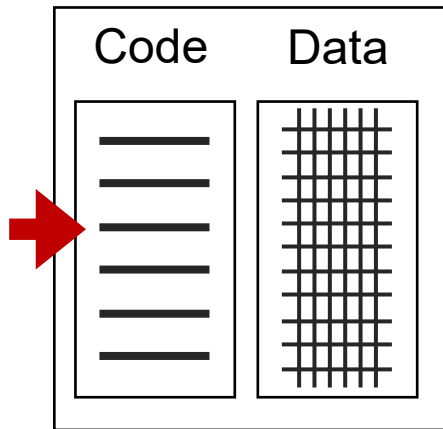
Outline

- Python numeric data types
- Python parallel processing

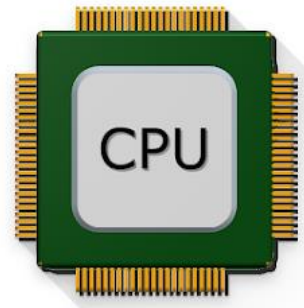
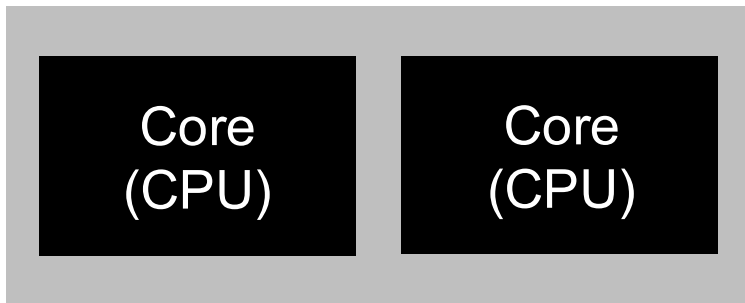
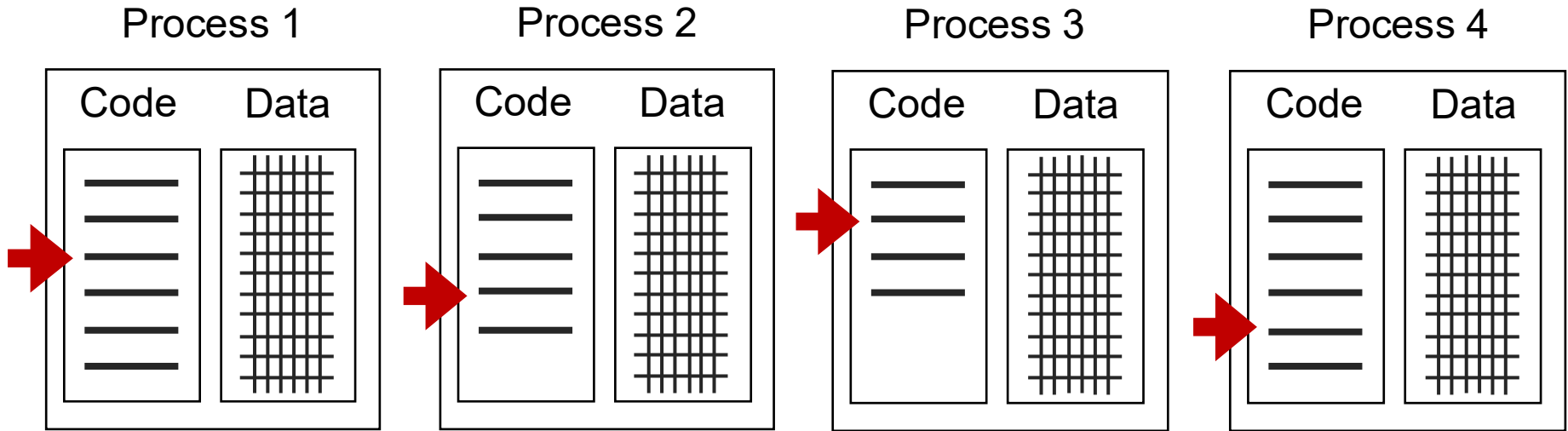


Instruction pointer
(also called “program counter”)

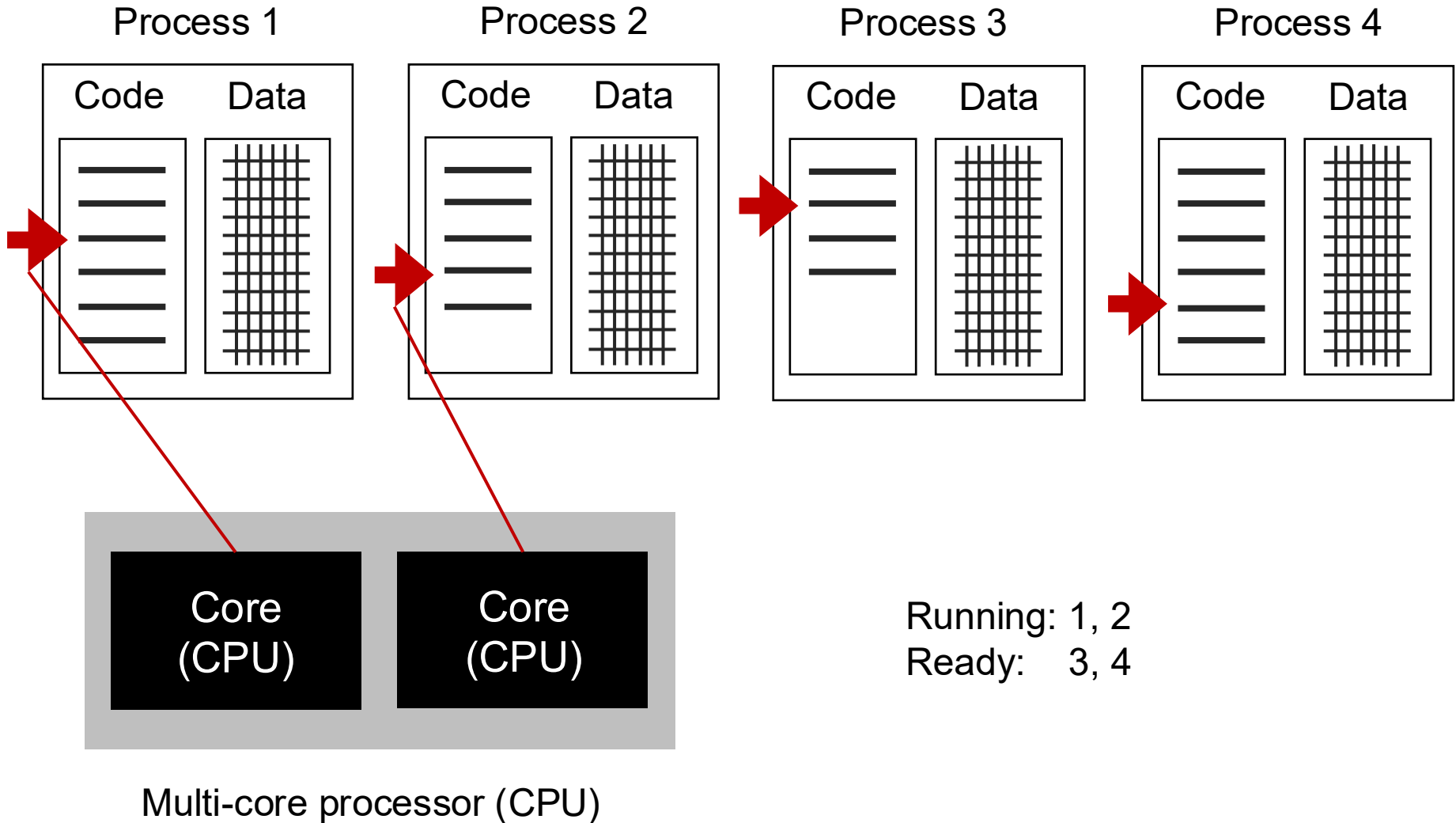
Process

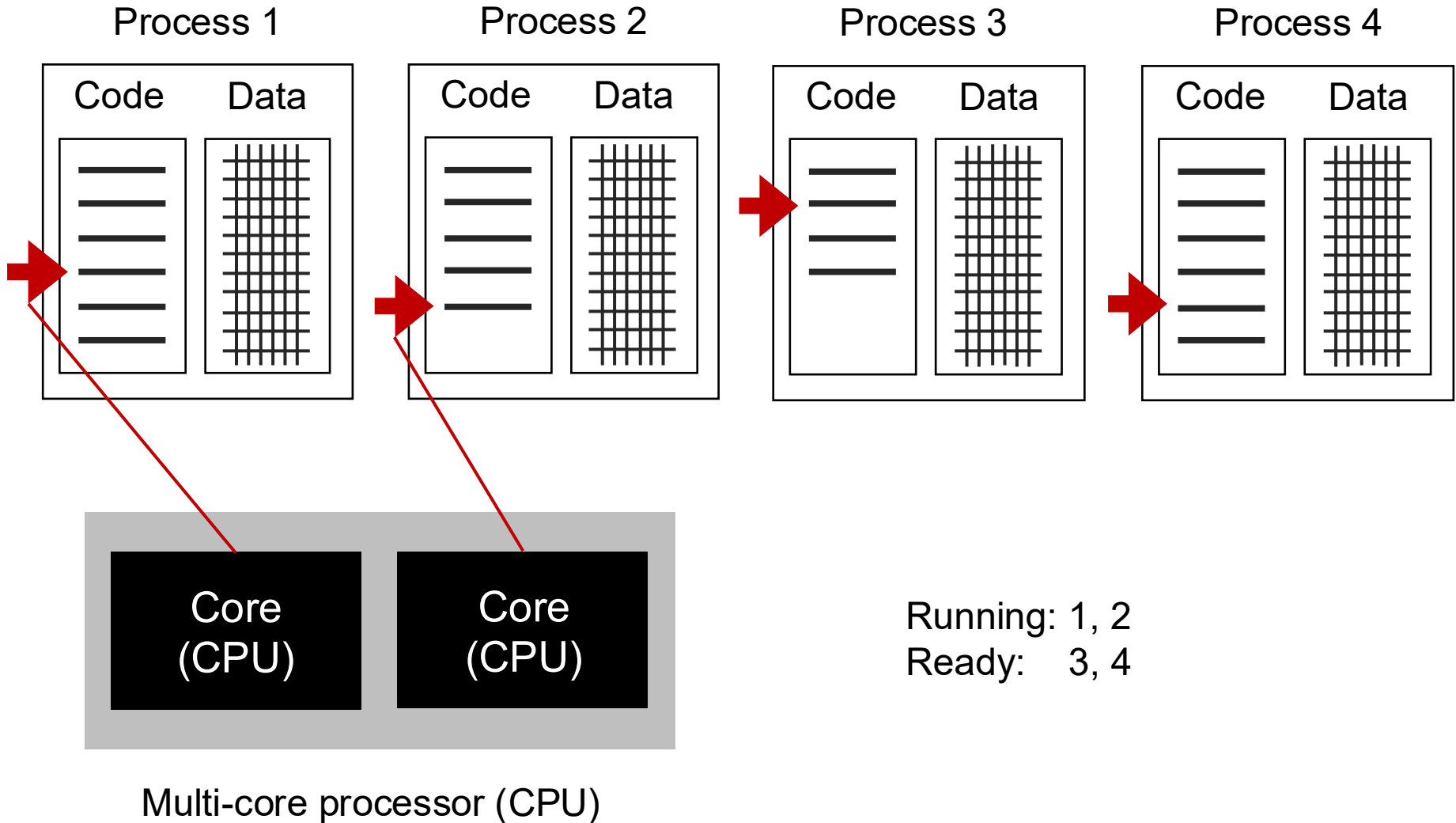


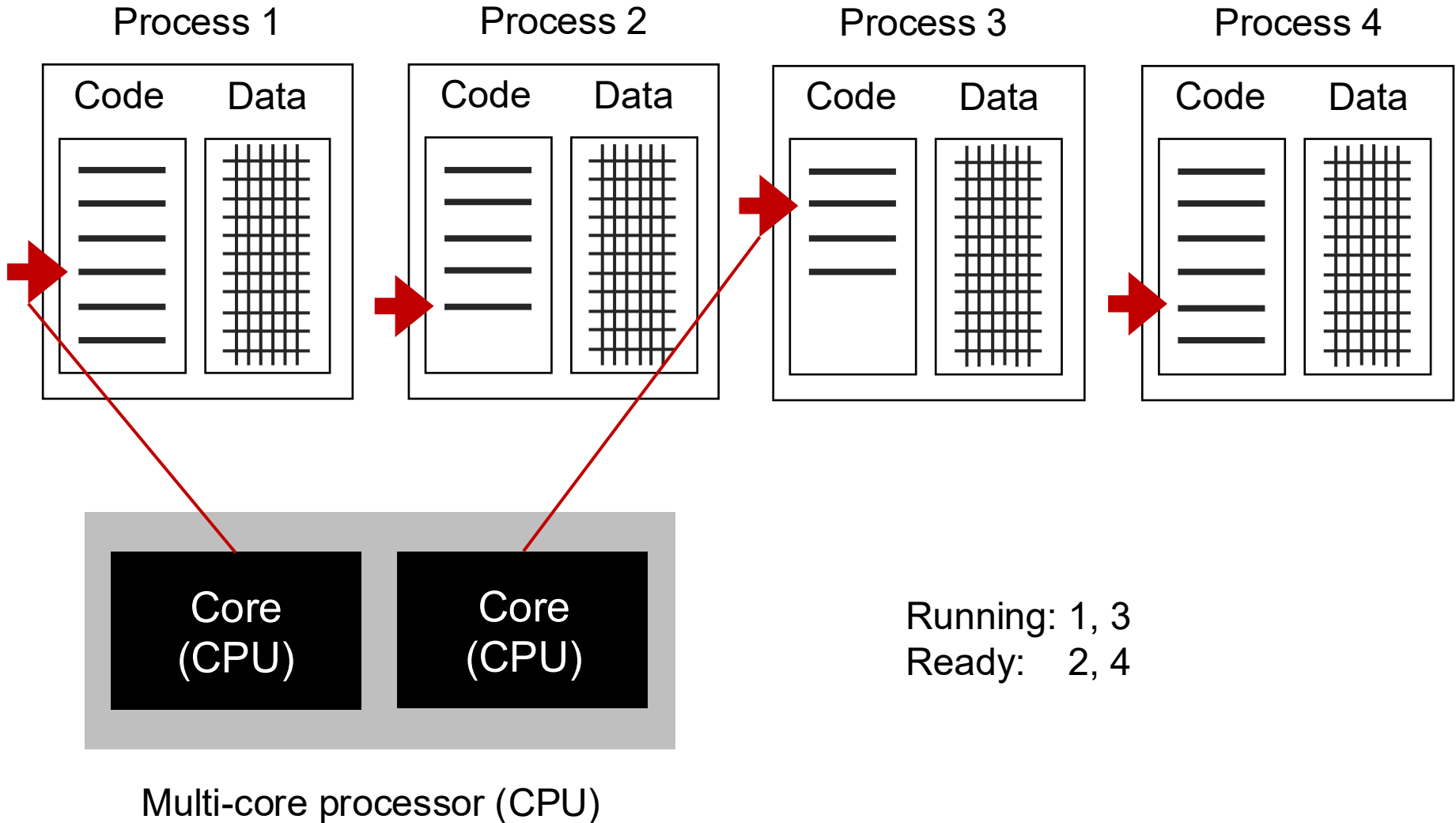
Instruction pointer belongs to a thread within the process

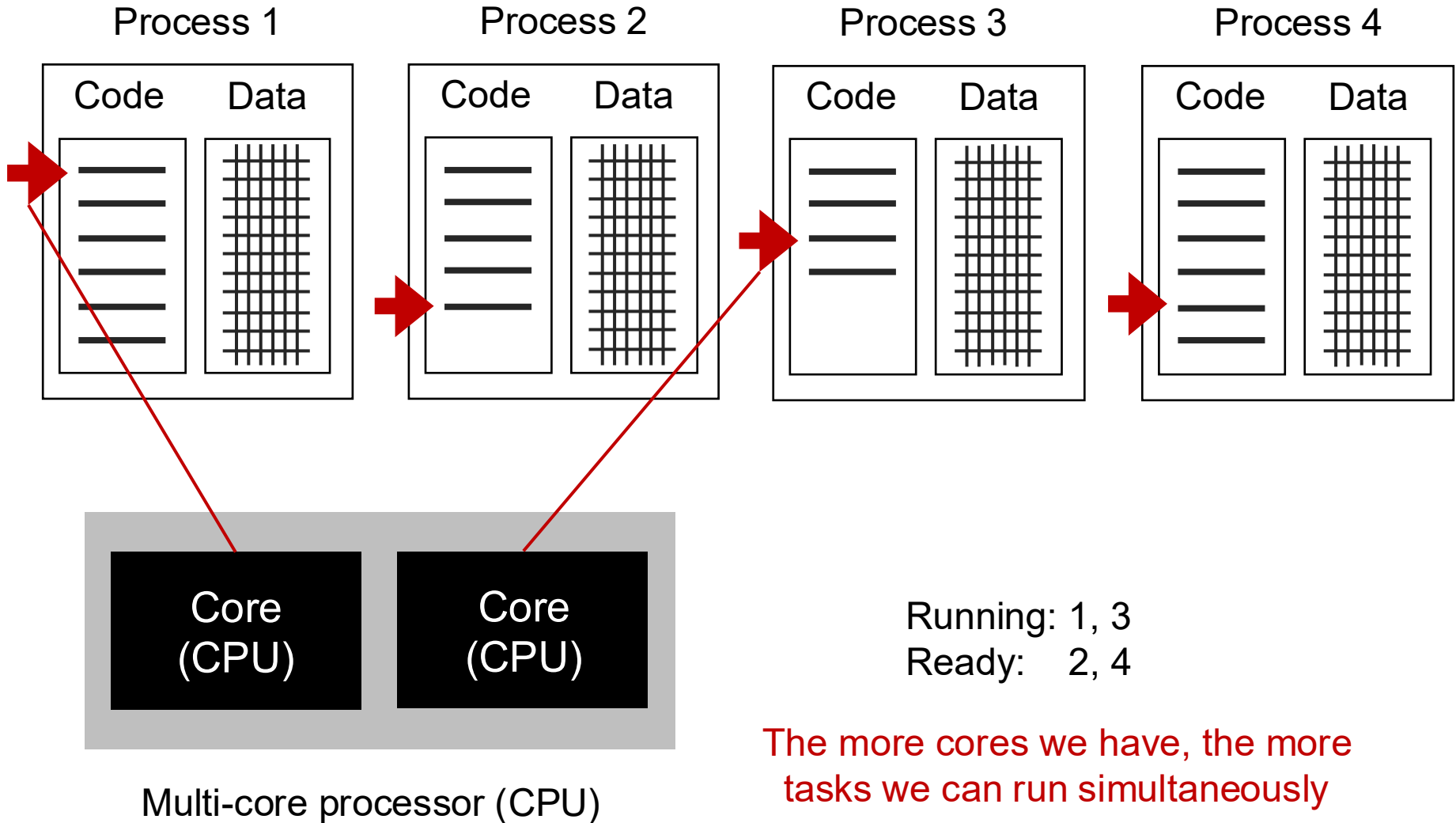


Multi-core processor (CPU)









Parallel execution models

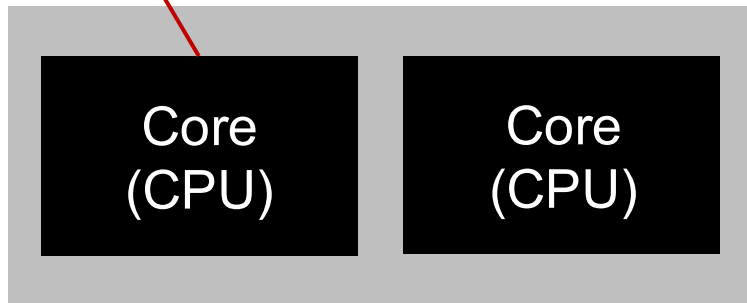
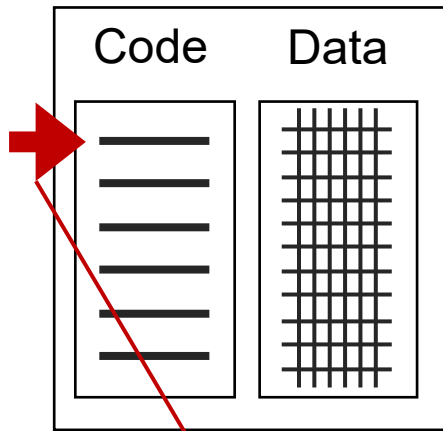
- Process-level parallelism
- Thread-level parallelism
- Task-level parallelism

Parallel execution models

- **Process-level parallelism**
- Thread-level parallelism
- Task-level parallelism

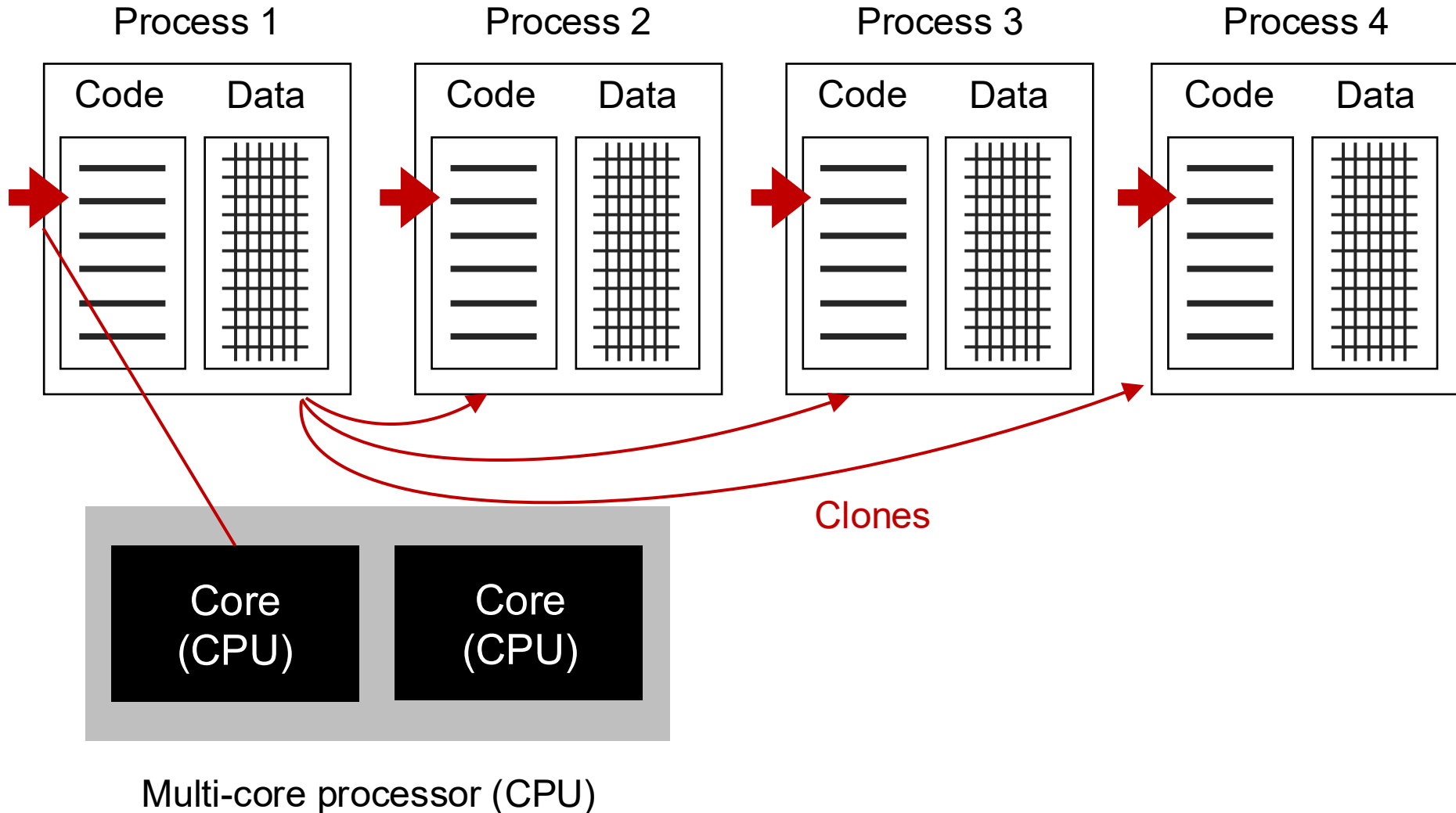
Process-level parallelism

Process 1

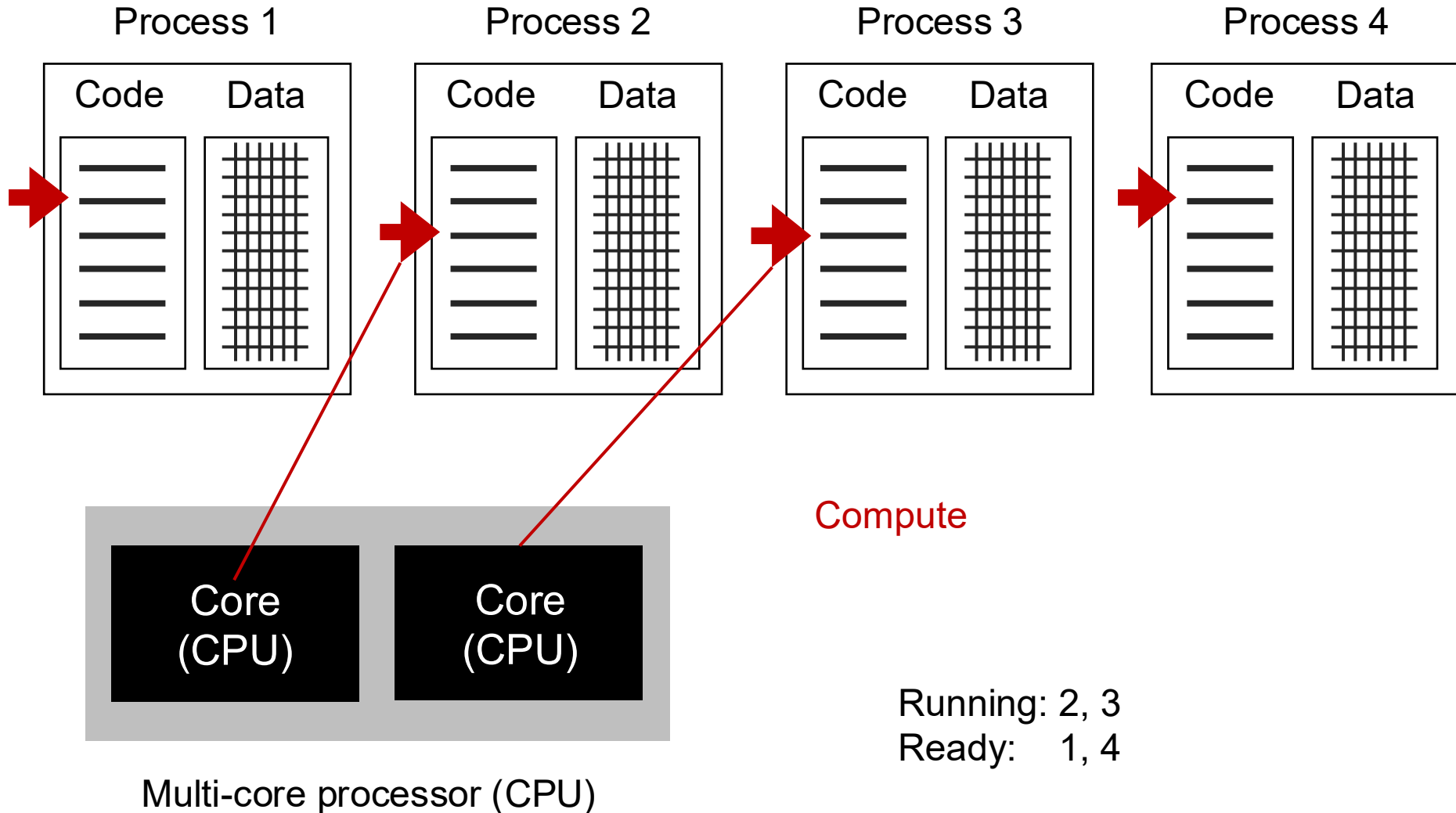


Multi-core processor (CPU)

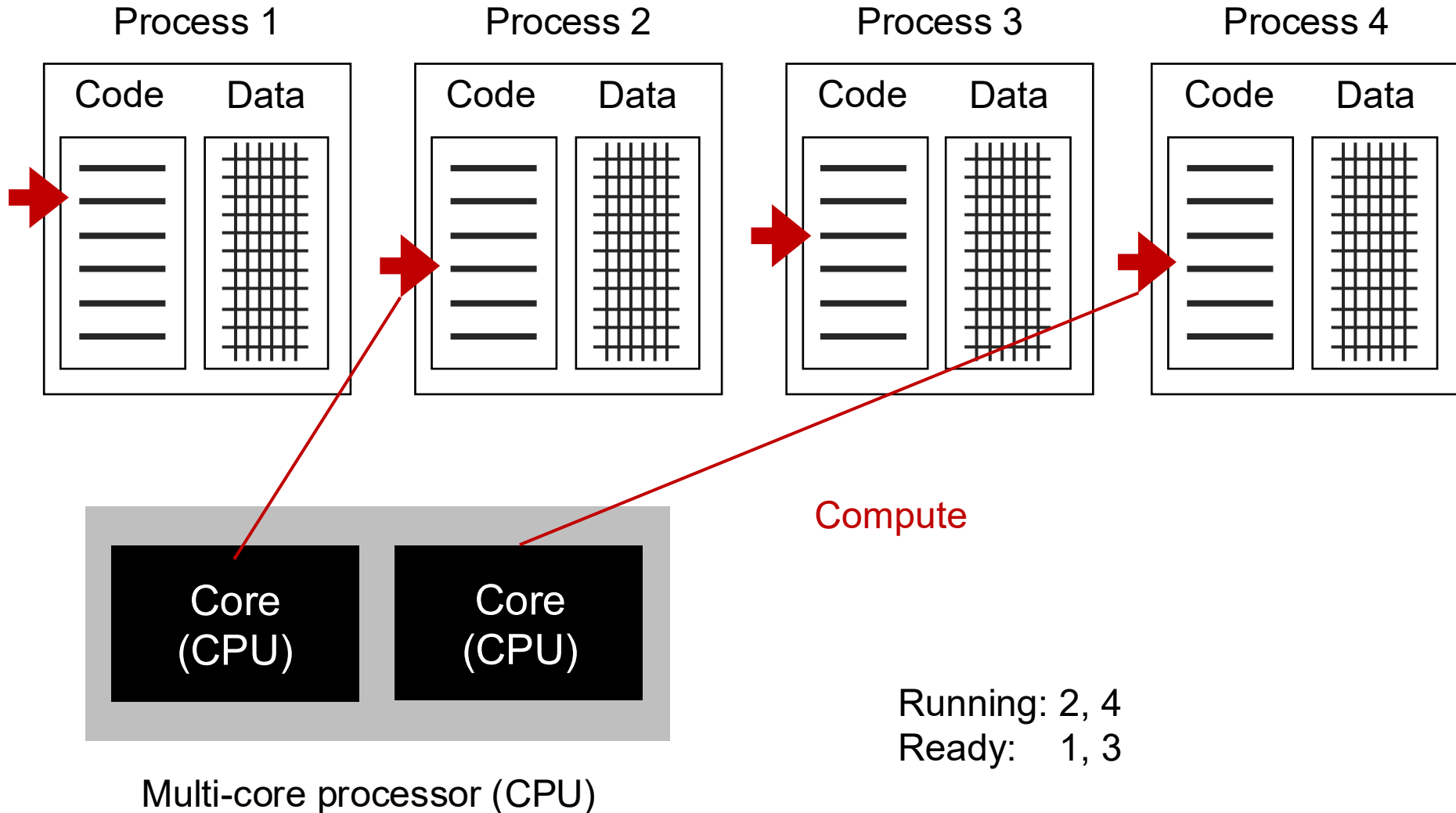
Process-level parallelism



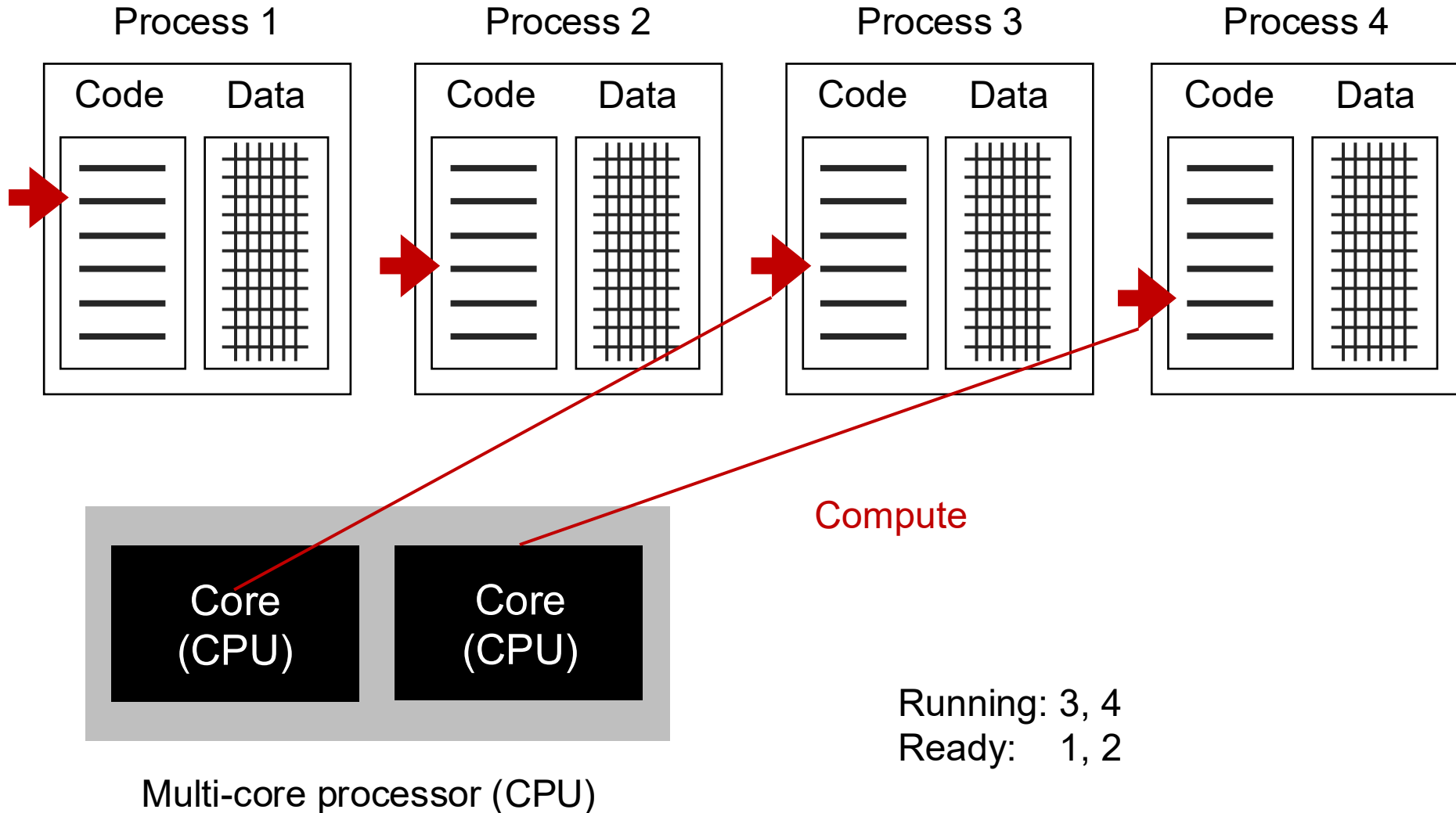
Process-level parallelism



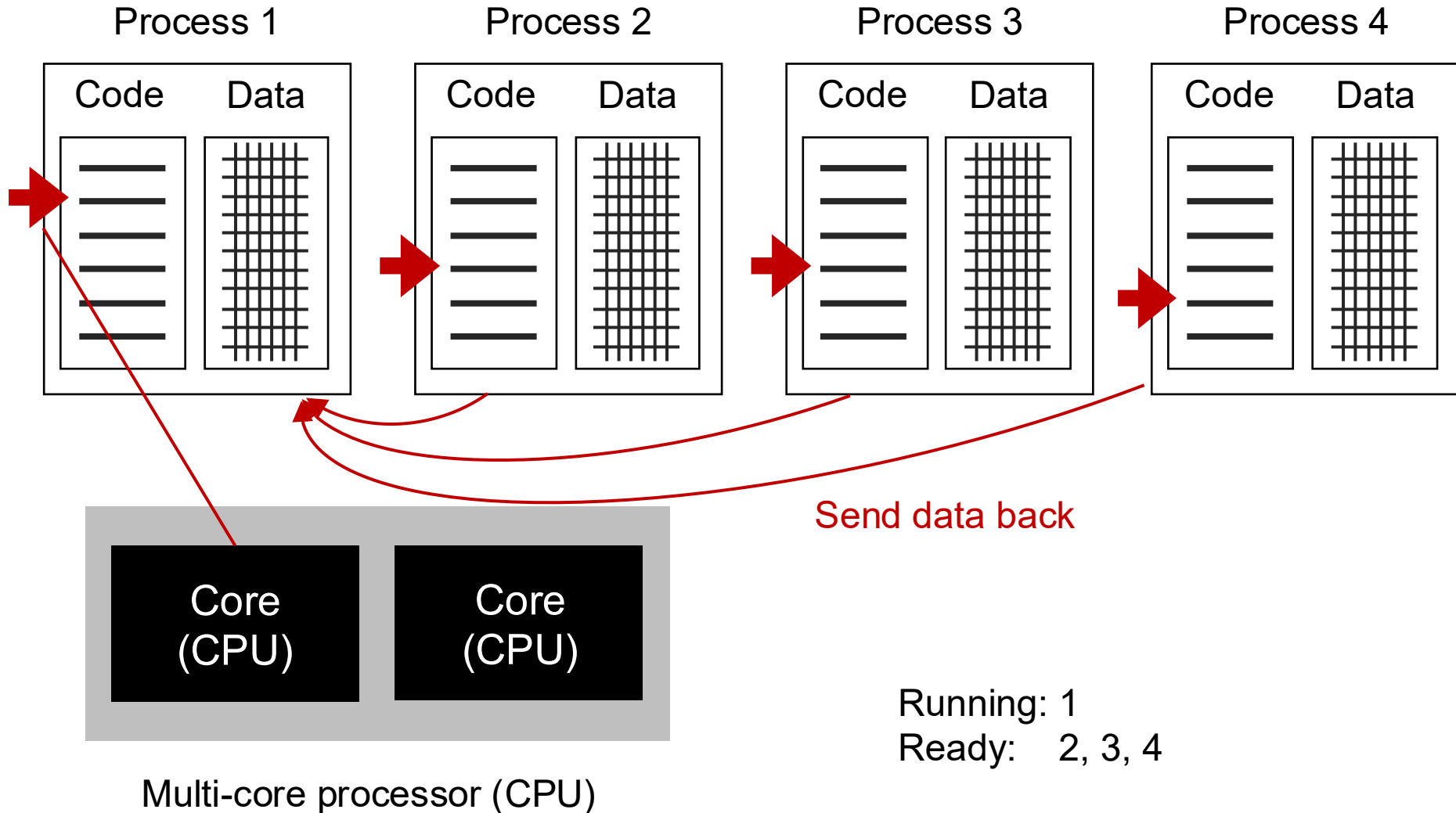
Process-level parallelism



Process-level parallelism

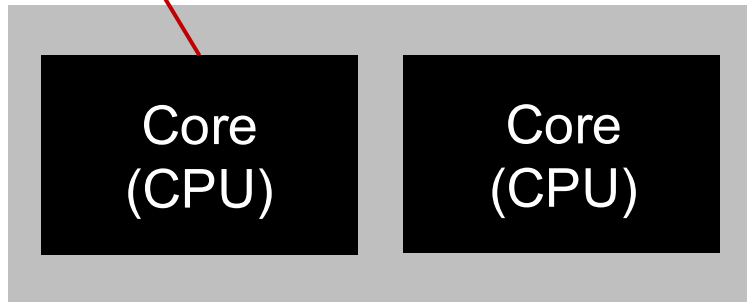
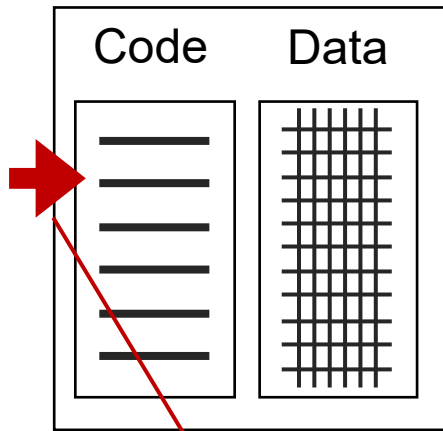


Process-level parallelism



Process-level parallelism

Process 1

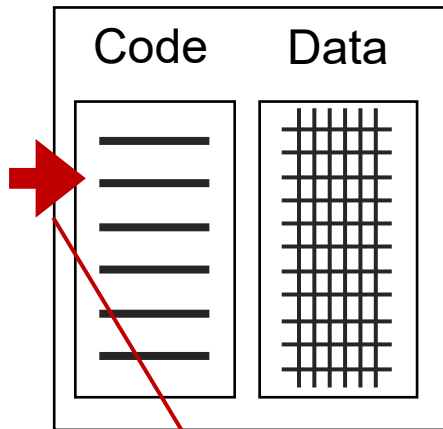


Multi-core processor (CPU)

Process-level parallelism in Python

Process 1

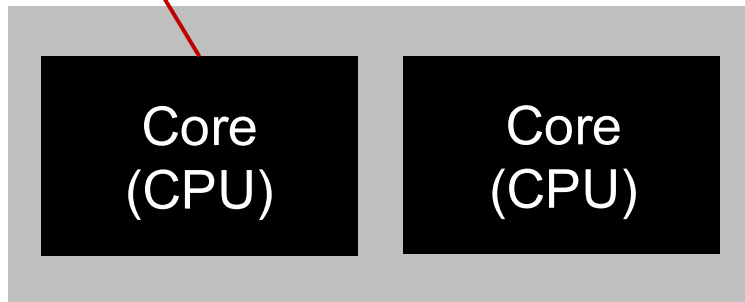
<https://docs.python.org/3/library/multiprocessing.html>



```
from multiprocessing import Pool

def f(x):
    return x*x

if __name__ == '__main__':
    with Pool(3) as p:
        print(p.map(f, [1,2,3]))
```

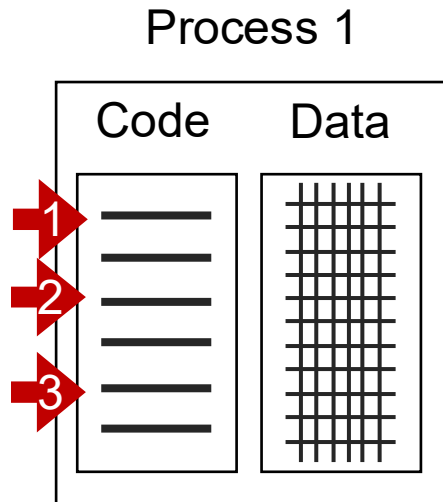


Multi-core processor (CPU)

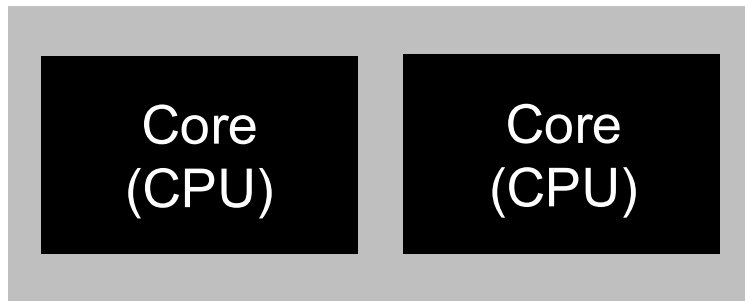
Parallel execution models

- Process-level parallelism
- **Thread-level parallelism**
- Task-level parallelism

Thread-level parallelism

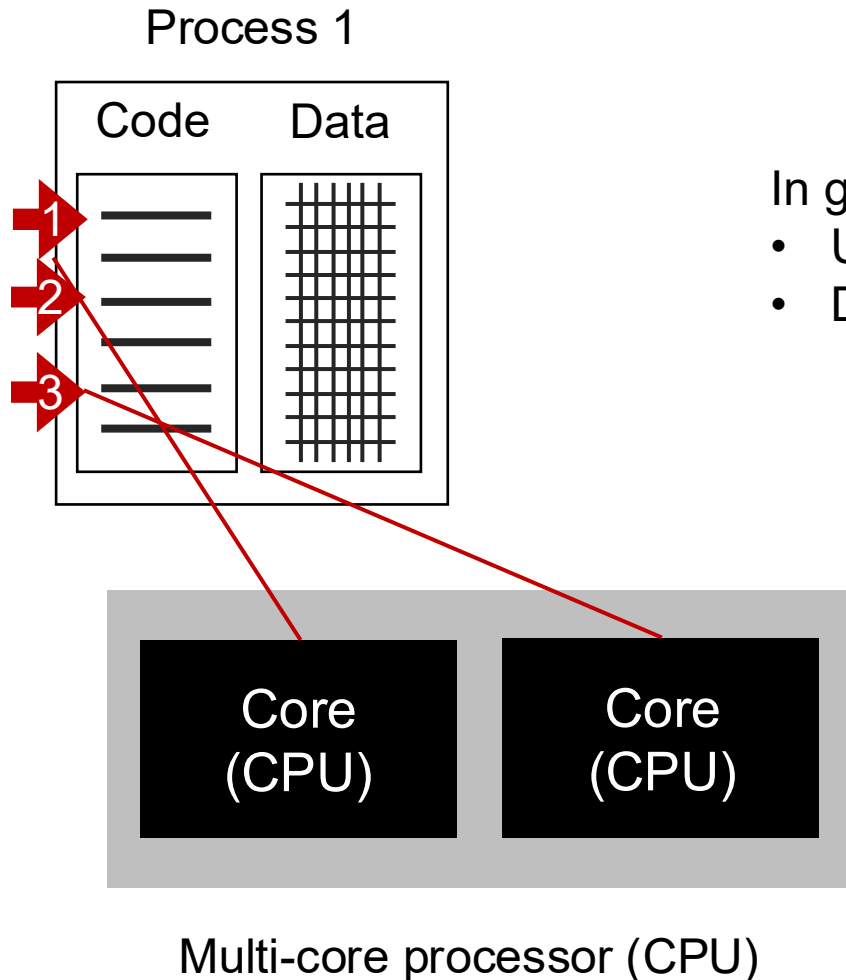


Threads give us multiple instruction pointers in a process, allowing us to execute multiple parts of the code at the same time!



Multi-core processor (CPU)

Thread-level parallelism

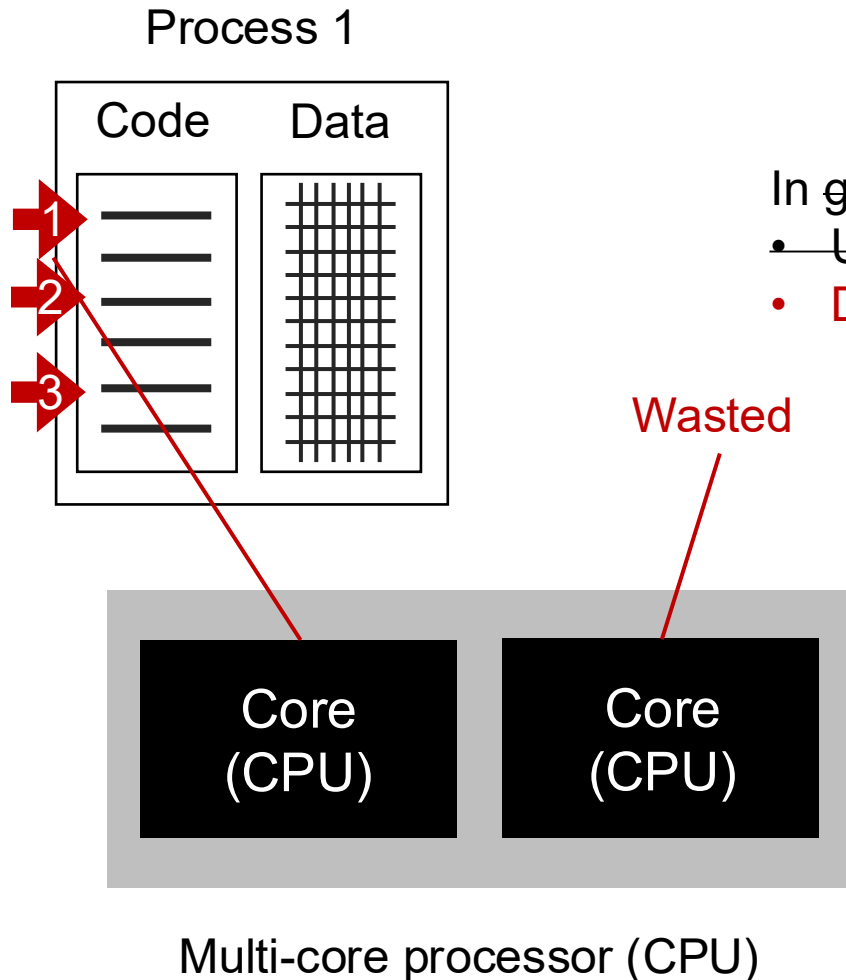


In general, threads help:

- Use multiple cores
- Do useful work when threads are blocking

Running: 1, 3
Ready: 2

Thread-level parallelism in Python: Before 3.13



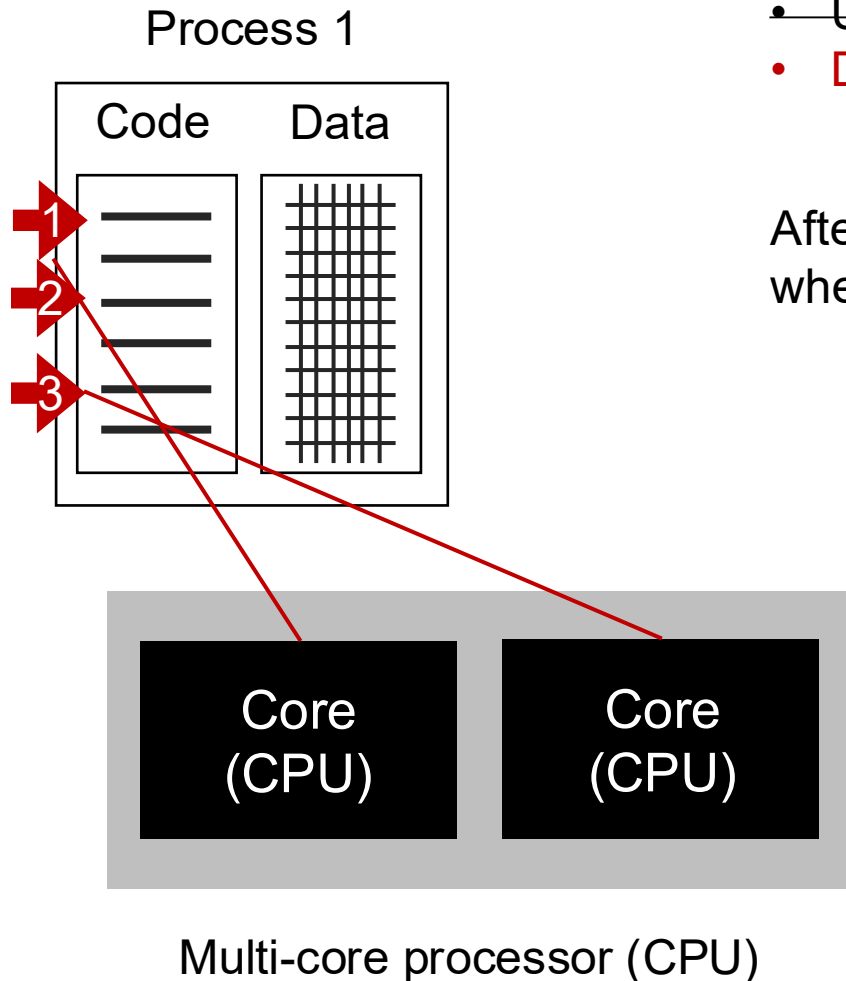
In general Python, threads help:

- ~~Use multiple cores~~ (b/c of the GIL)
- Do useful work when threads are blocking

<https://wiki.python.org/moin/GlobalInterpreterLock>

Running: 1
Ready: 3
Blocked: 2

Thread-level parallelism in Python: Starting with 3.13



Before 3.13, threads help:

- ~~Use multiple cores (b/c of the GIL)~~
- Do useful work when threads are blocking

<https://wiki.python.org/moin/GlobalInterpreterLock>

After 3.13, CPython supports free threading where GIL is disabled.

<https://docs.python.org/3/howto/free-threading-python.html>

Running: 1, 3

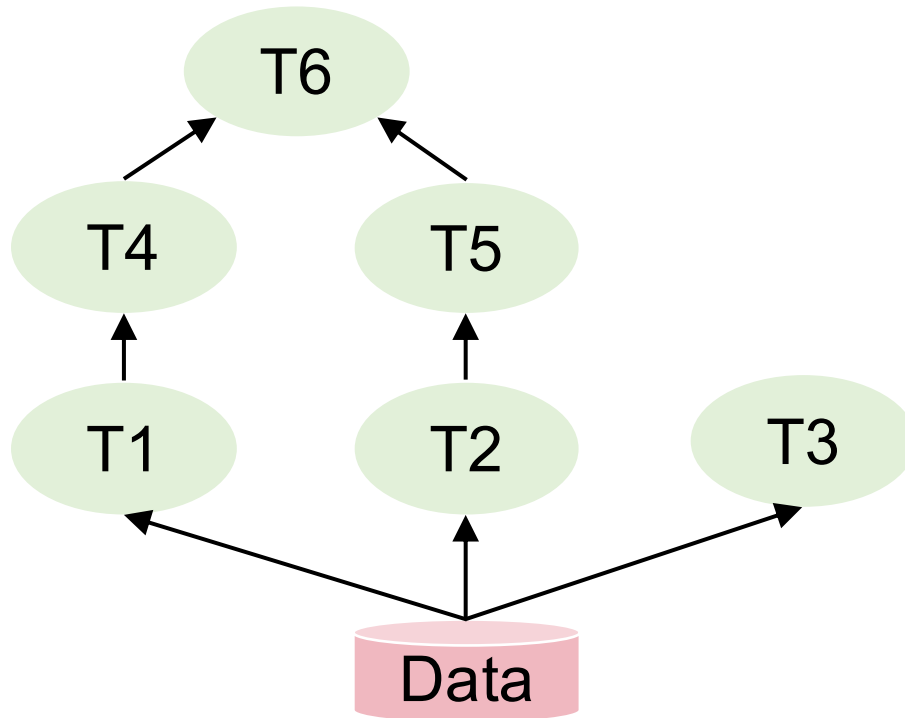
Ready:

Blocked: 2

Parallel execution models

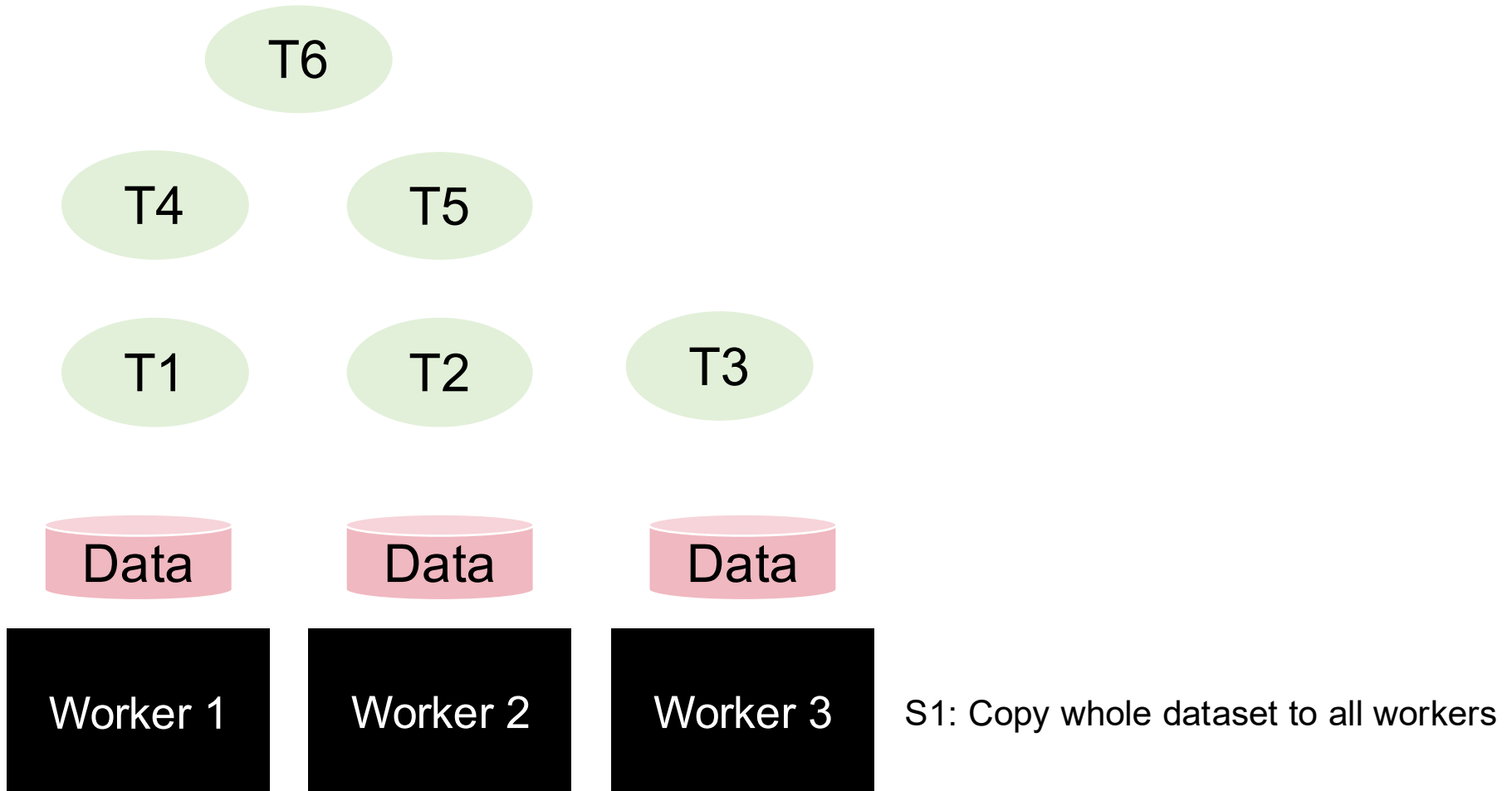
- Process-level parallelism
- Thread-level parallelism
- **Task-level parallelism**

Task-level parallelism

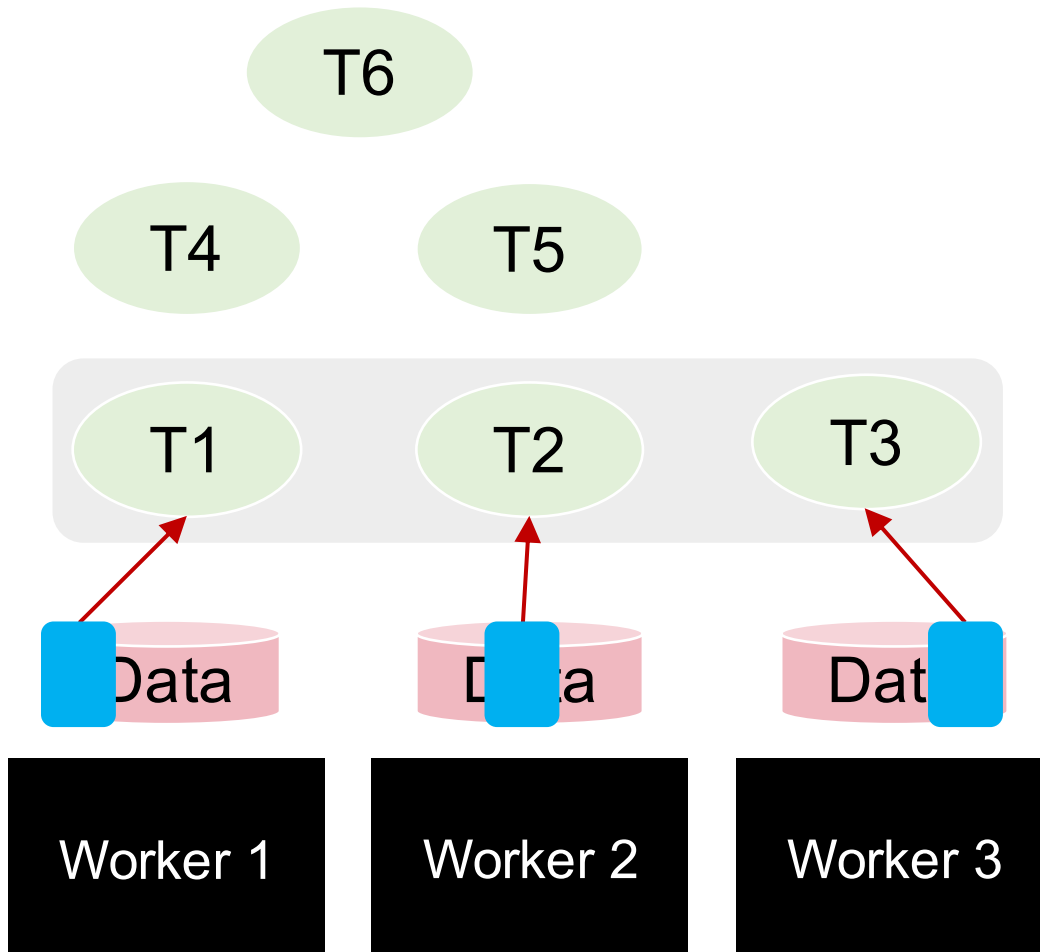


Task DAG
(Directed Acyclic Graph)

Task-level parallelism



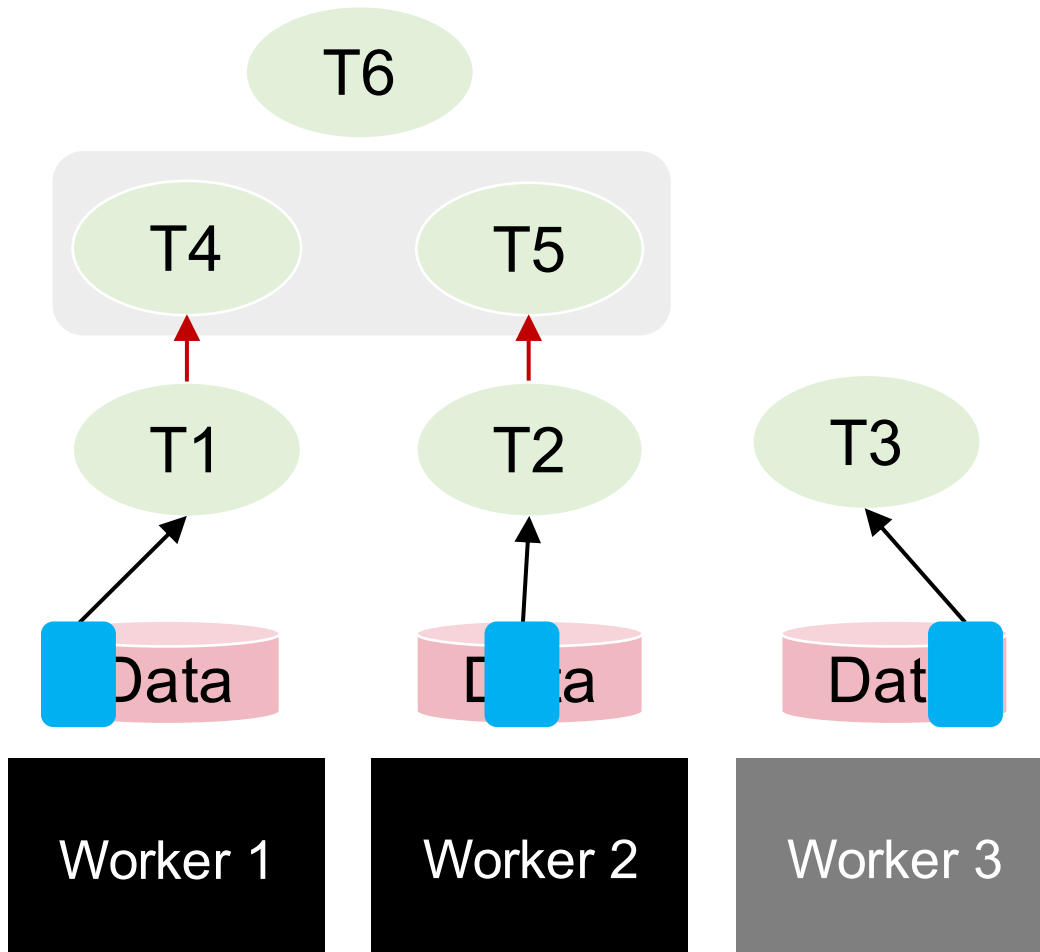
Task-level parallelism



S2: Schedule T1 to W1, T2 to W2, T3 to W3

S1: Copy whole dataset to all workers

Task-level parallelism

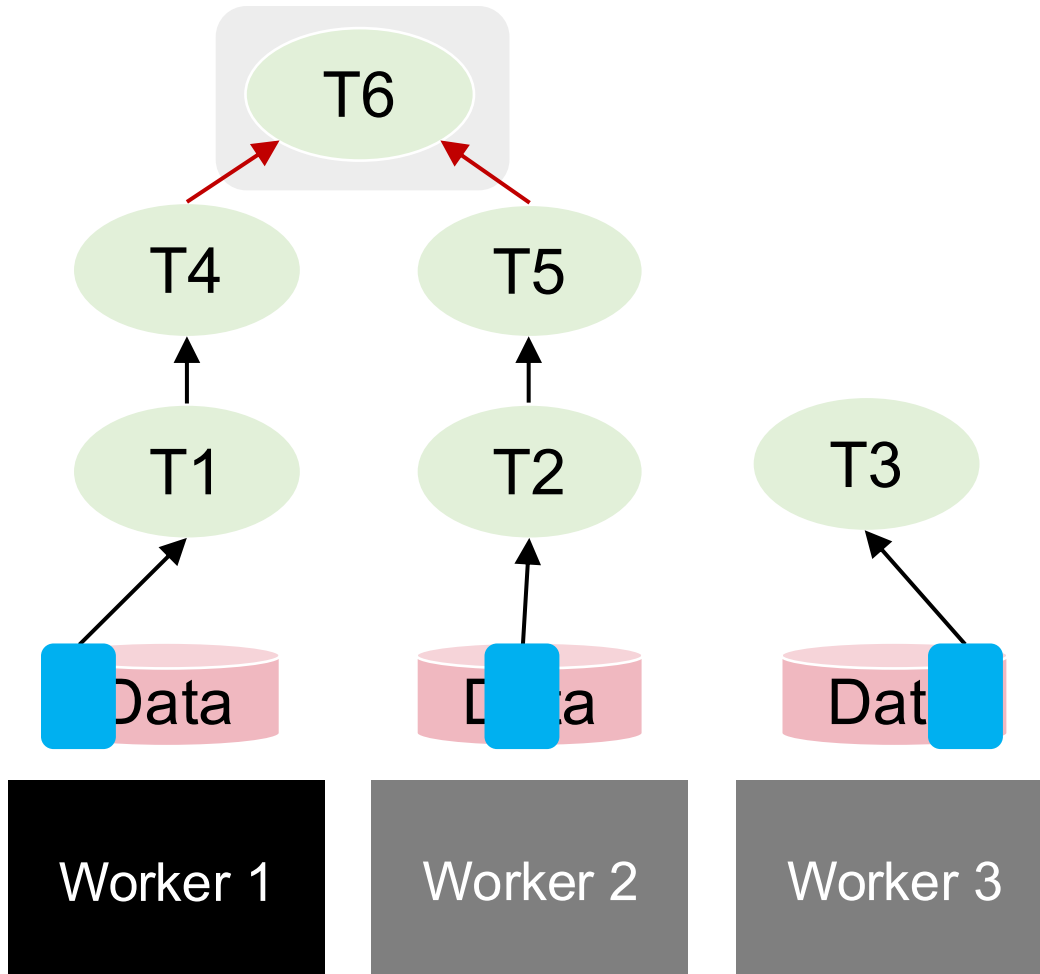


S3: Run T4 after T1 on W1, run T5 after T2 on W2; after T3, W3 is idle

S2: Schedule T1 to W1, T2 to W2, T3 to W3

S1: Copy whole dataset to all workers

Task-level parallelism



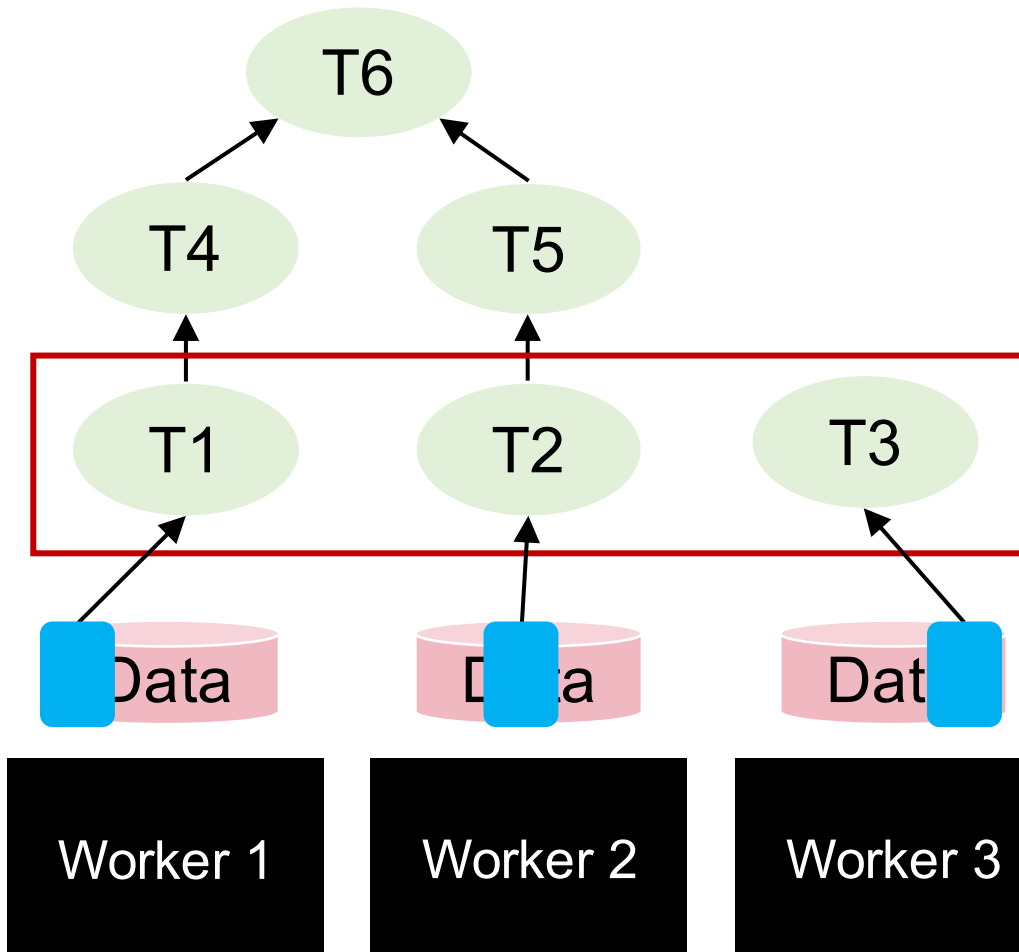
S4: After T4 and T5 ends, run T6 on W1; after T5, W2 is idle

S3: Run T4 after T1 on W1, run T5 after T2 on W2; after T3, W3 is idle

S2: Schedule T1 to W1, T2 to W2, T3 to W3

S1: Copy whole dataset to all workers

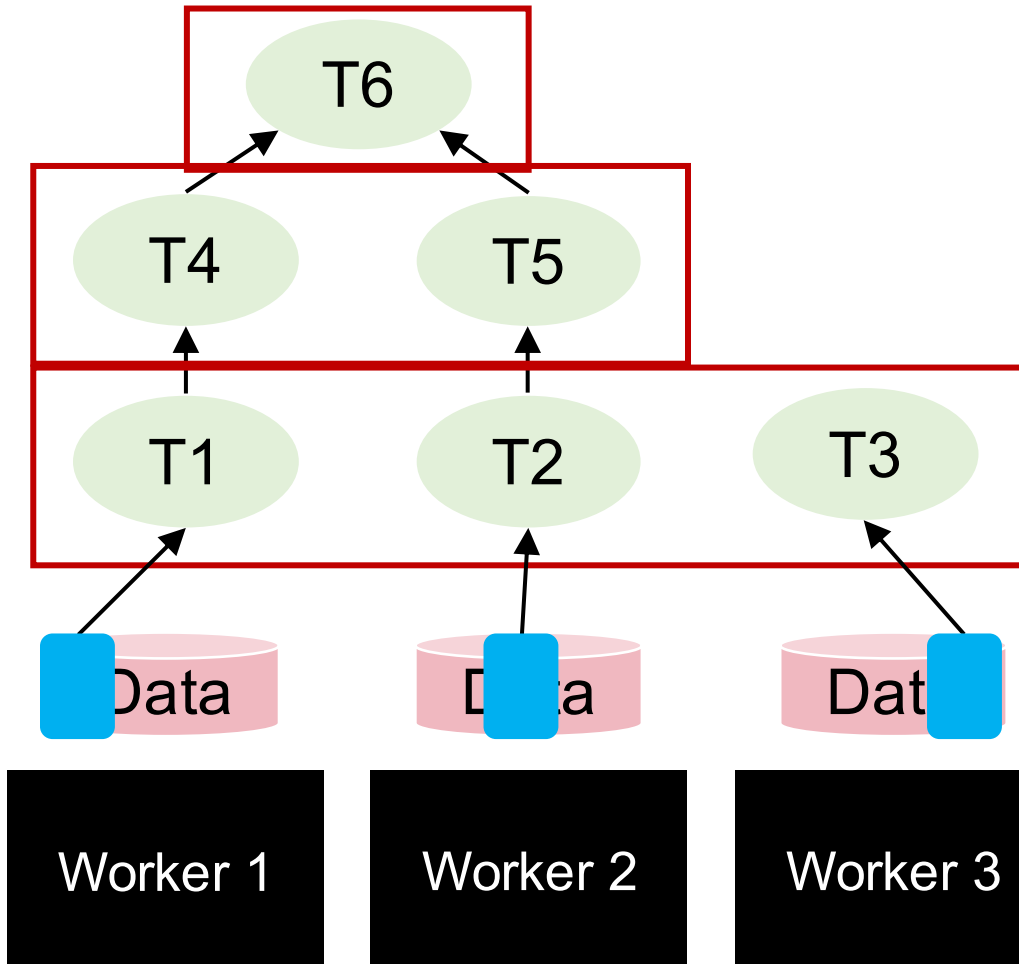
Task-level parallelism



Degree of parallelism is the largest amount of parallelism possible in the DAG:

- How many tasks can be run in parallel at most

Task-level parallelism



Observations:

Resource wastage on idle workers

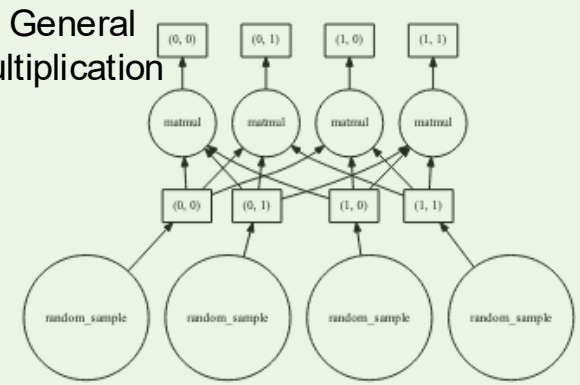
Overtime degree of parallelism drops!

Degree of parallelism is the largest amount of parallelism possible in the DAG:

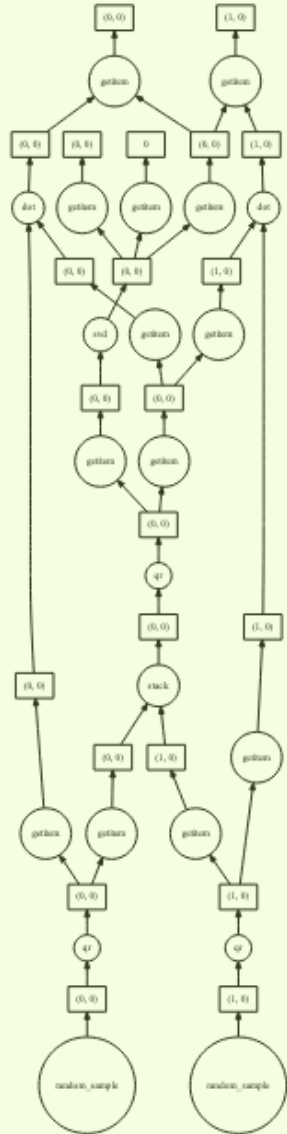
- How many tasks can be run in parallel at most

Parallel program DAGs

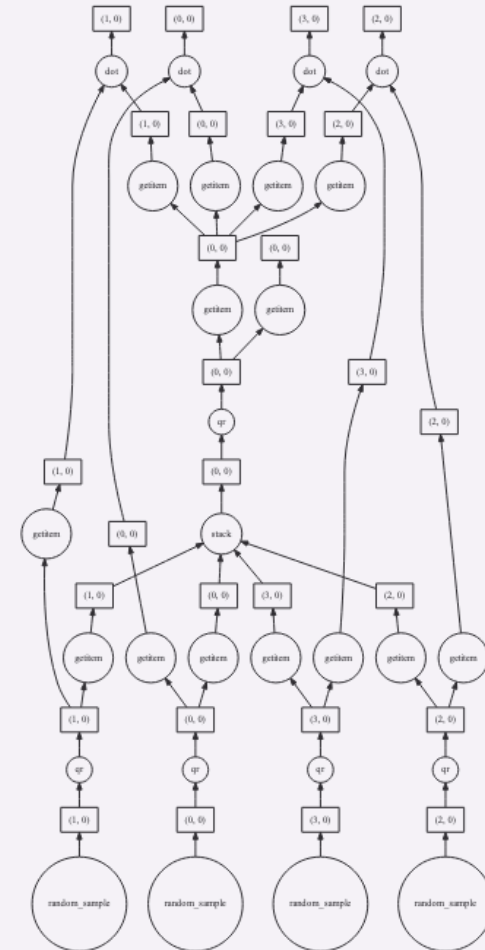
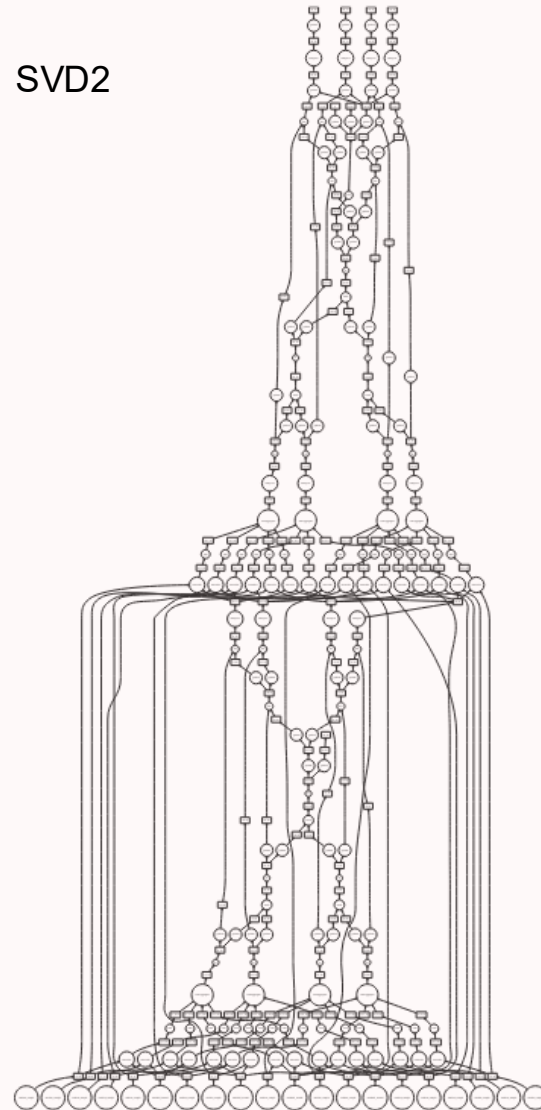
GEMM: General
Matrix Multiplication



SVD1: Singular Value Decomposition



SVD2



TSQR: Tall-and-Skinny
QR
Factorization

Quantify benefit of parallelism: Speedup

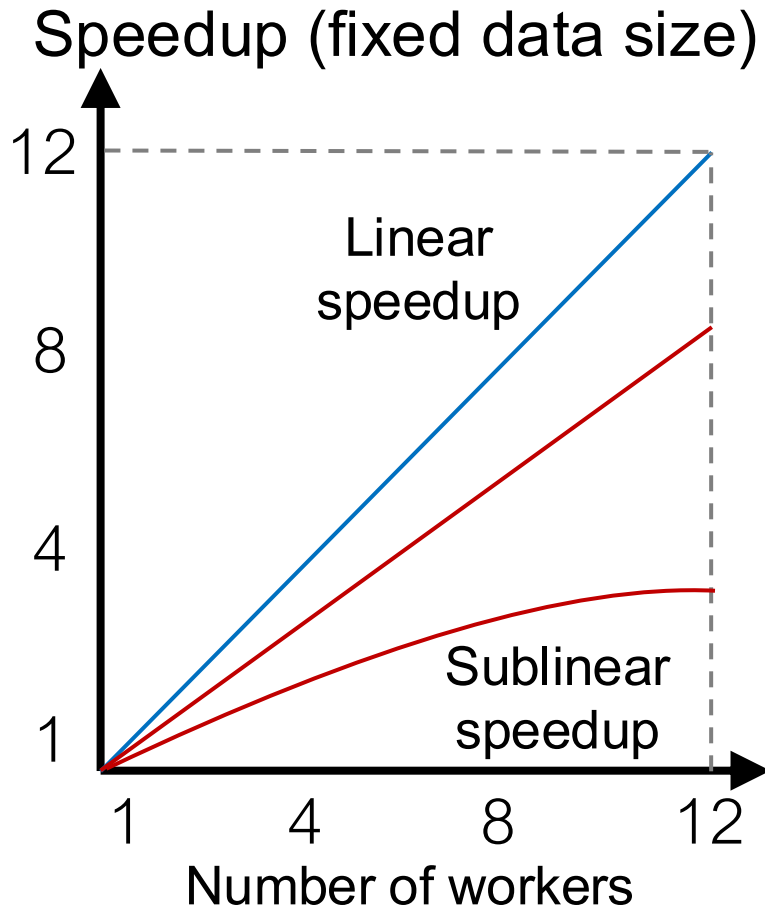
$$\text{Speedup} = \frac{\text{Completion time given 1 worker}}{\text{Completion time given } N \text{ worker}}$$

Quantify benefit of parallelism: Speedup

$$\text{Speedup} = \frac{\text{Completion time given 1 worker}}{\text{Completion time given } N \text{ worker}}$$

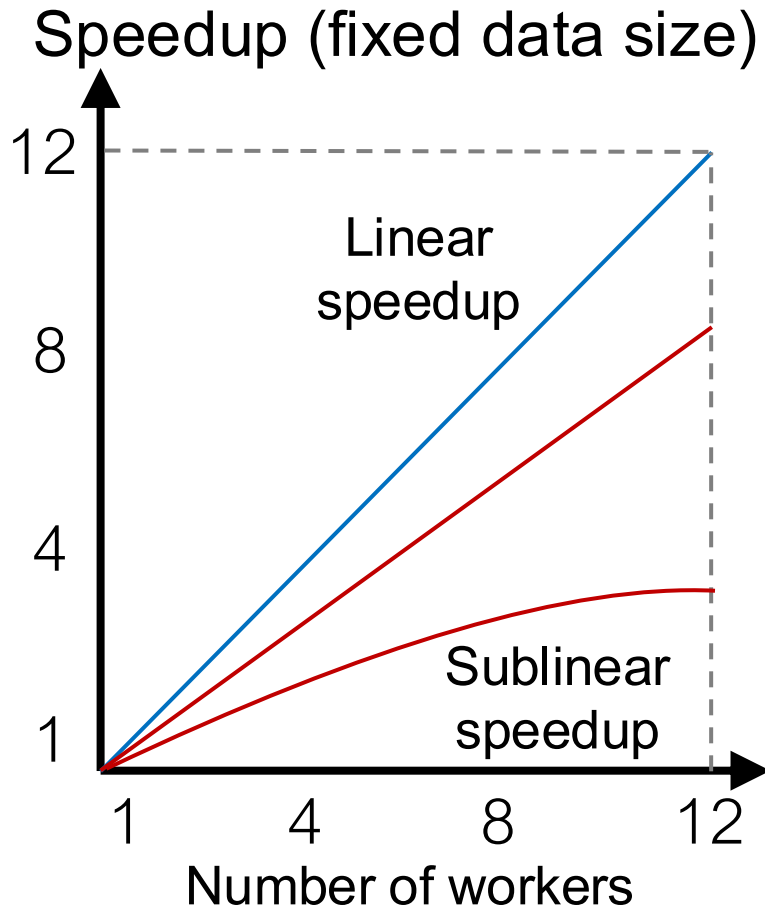
Q: Given N workers, can we get a speedup of N ?

Quantify speedup

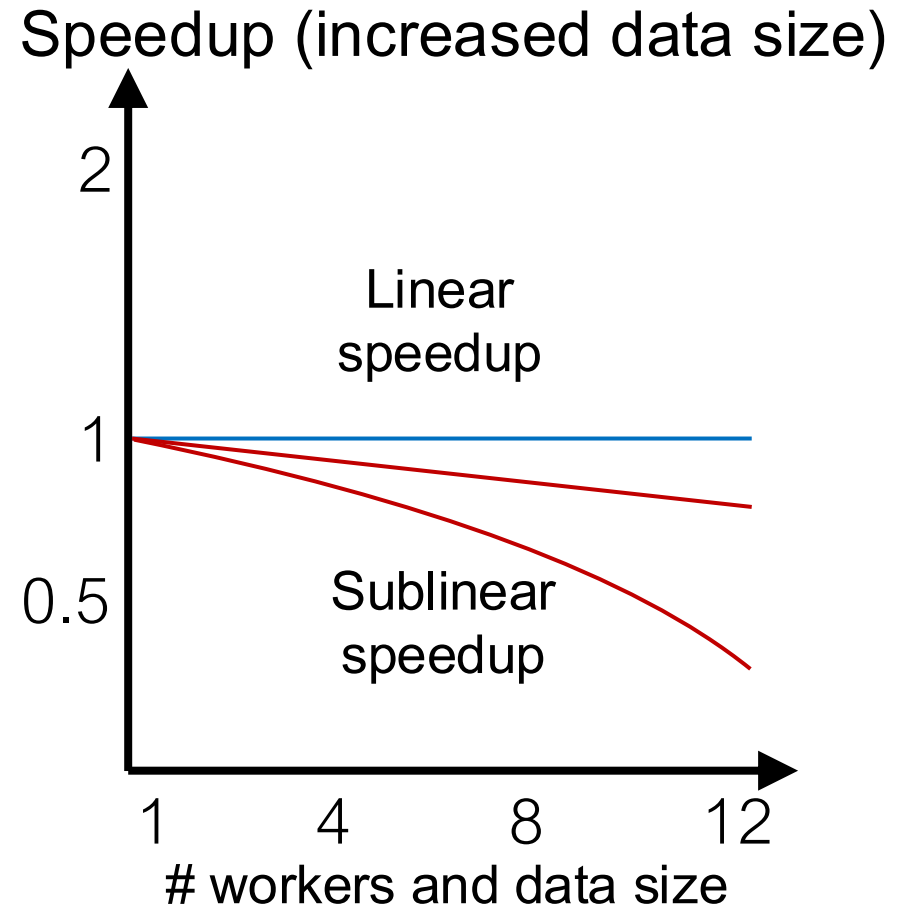


Strong scaling

Quantify speedup

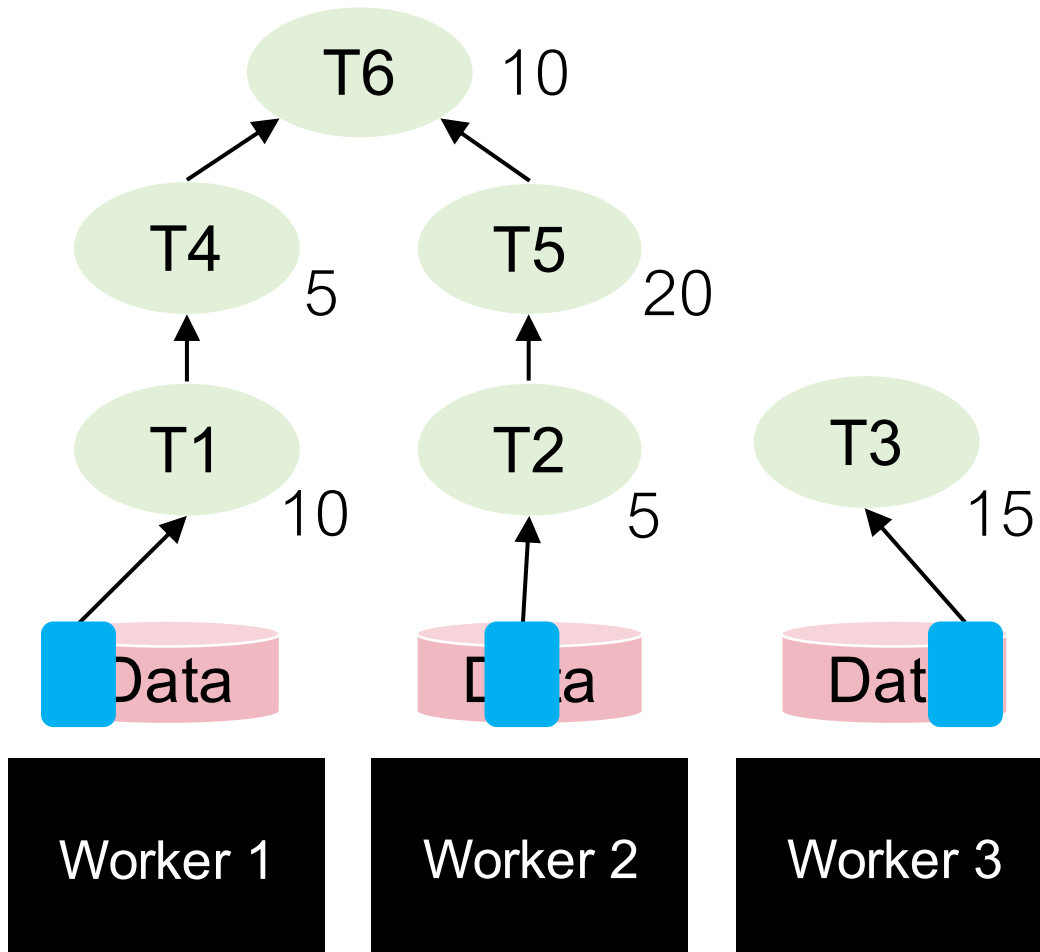


Strong scaling



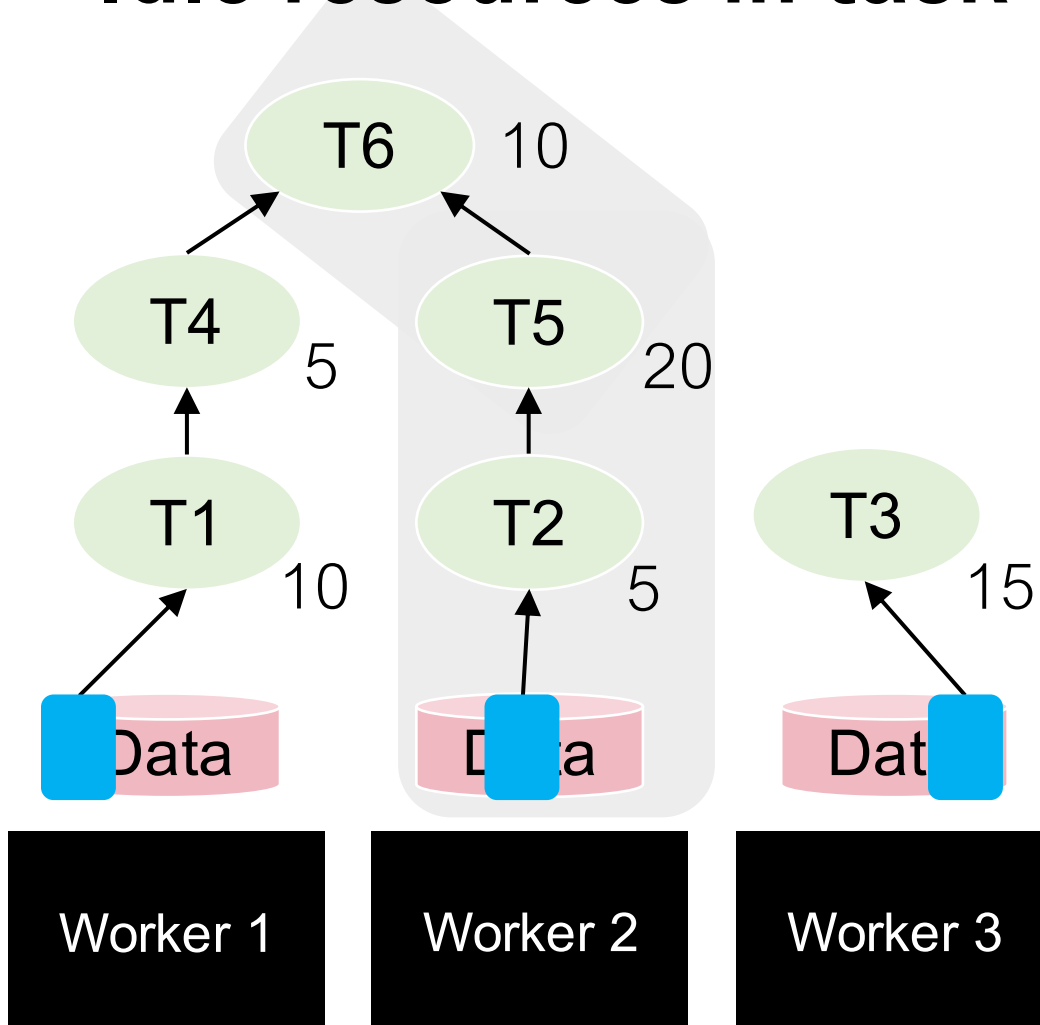
Weak scaling

Idle resources in task-level parallelism



Task completion time varies

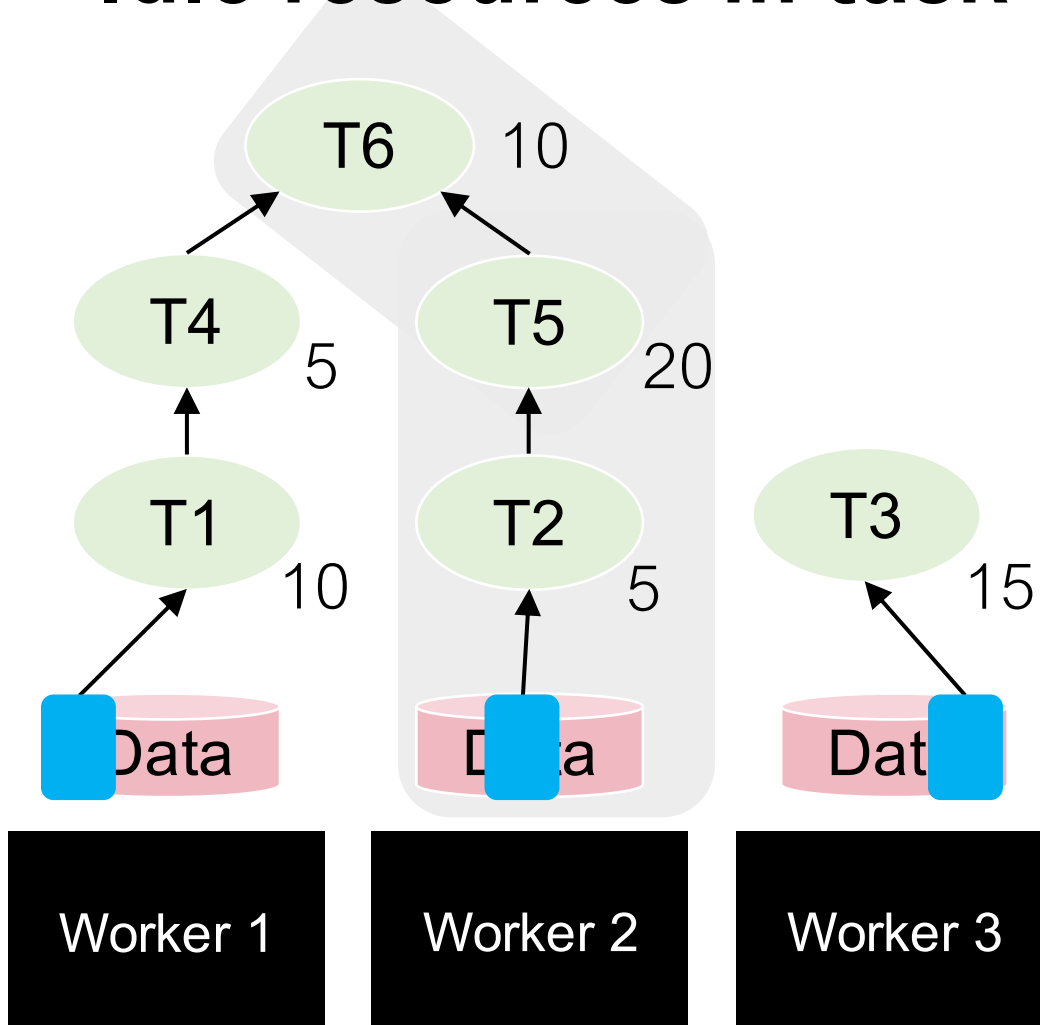
Idle resources in task-level parallelism



- Job completion time is always bounded by the **longest path** in the DAG

Task completion time varies

Idle resources in task-level parallelism



- Job completion time is always bounded by the **longest path** in the DAG
- **Potential optimization:** The scheduler can elastically release a worker if it knows the worker will be idle till the end
 - Can **save \$ cost** in cloud

Task completion time varies

Idle resources in task-level parallelism

Q: What's the job completion time with 1 worker?

Q: What's the job completion time with 3 workers?

Q: What's the speedup?

